

## CHAPTER 9

# Stack and Subroutines

---

## The 8085 Microprocessor

Reference: Ramesh S. Gaonkar — Microprocessor Architecture, Programming, and Applications with the 8085 (5th Edition)

### SUBMITTED BY

|                              |          |
|------------------------------|----------|
| Md. Arif Shekh               | 23103022 |
| Md. Meherab Hossain Talukder | 23103032 |
| Tasfia Islam Prapty          | 23103286 |
| G. M. Imtiaz Dinar           | 23103080 |
| Tasnia Chowdhury Toshita     | 23103038 |

### SUBMITTED TO

**Dr. Md. Alomgir Hossain**

Associate Professor, Dept. of CSE  
IUBAT

# What This Chapter Covers

## 9.1 Stack

Where the 8085 puts things down

## 9.2 Subroutine

How the 8085 takes a detour

## 9.3 Restart & Conditional Call / Return

Taking the detour only if...

## 9.4 Advanced Subroutine Concepts

Detours inside detours

# The Stack — Memory, Not a Register

## STACK

- Group of memory locations
- In R/W memory (RAM)
- Temporary storage of binary data
- Defined by the programmer
- Reserved at the high end of memory

## STACK POINTER

- 16-bit register inside the 8085
- Holds a memory address
- Always points to the top
- $SP - 2$  on PUSH
- $SP + 2$  on POP

**Similar names — completely different things**

# Defining the Stack — LXI SP

```
LXI SP, 2099H
```

- 3-byte instruction
- Loads the stack pointer register
- Beginning of the stack is defined
- Stack can start anywhere in memory

## WHAT ACTUALLY HAPPENS

SP = 2099H

loaded

First byte → 2098H

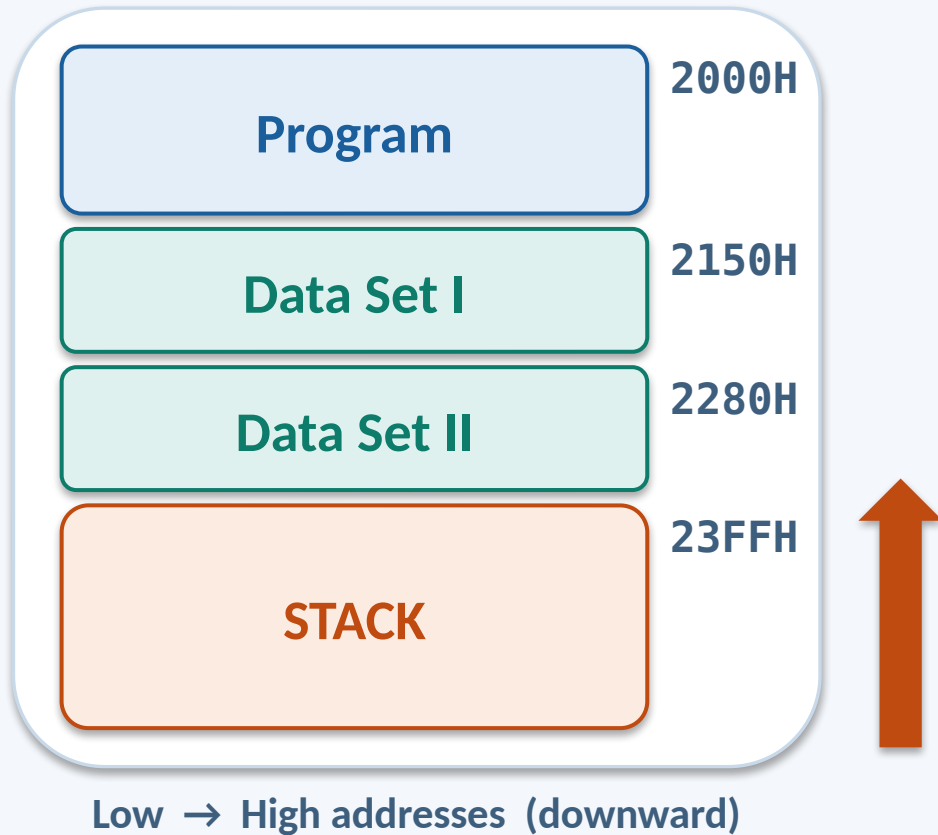
one less

Then 2097H, 2096H ...

decreasing order

**Rule: initialize at the highest location**

# How the Stack Grows



- Grows upward on the memory map
- Addresses go downward (2098H, 2097H ...)
- PUSH → SP decreases
- POP → SP increases
- Must never overlap the program
- R/W memory holds: program · data · stack

# PUSH — Store Register Pair on Stack

## OPERANDS

**PUSH B**

B-C

**PUSH D**

D-E

**PUSH H**

H-L

**PUSH PSW**

A + Flags

- 1-byte instruction
- SP is decremented → high-order register stored
- SP is decremented again → low-order register stored
- Stack pointer decremented by 2 in total
- Copy operation — the source is not modified
- No flags are affected

**The programmer's way to save a register pair before it is reused**

# POP — Retrieve Register Pair from Stack

## OPERANDS

**POP B**

B-C

**POP D**

D-E

**POP H**

H-L

**POP PSW**

A + Flags

- 1-byte instruction
- Top location → low-order register,  $SP + 1$
- Next location → high-order register,  $SP + 1$
- Stack pointer incremented by 2 in total
- Copies the top two locations of the stack
- No flags are affected

**PSW = Accumulator + Flag Register — PUSH PSW / POP PSW is the only way to reach the flags**

# PUSH H and POP H in Action

| ADDRESS | INSTRUCTION          | EFFECT                |
|---------|----------------------|-----------------------|
| 2000    | <b>LXI SP, 2099H</b> | Initialize the stack  |
| 2003    | <b>LXI H, 42F2H</b>  | H = 42H L = F2H       |
| 2006    | <b>PUSH H</b>        | Save HL on the stack  |
| 2007    | <b>DELAY COUNTER</b> | HL is free to be used |
| 2010    | <b>POP H</b>         | Restore HL            |

## THE STACK

2099

SP starts here

2098

42H ← (H)

2097

F2H ← (L)

POP H → SP back to 2099

LIFO — Last In, First Out    PUSH H · B · PSW   →   POP PSW · B · H

# Key Rules of the Stack

- 1 Stack = memory locations • Stack pointer = register
- 2 R/W memory holds program • data • stack
- 3 Stack grows in decreasing memory addresses
- 4 Initialize at the highest user location
- 5 SP points to the top; storage begins one below
- 6 Always store and retrieve in LIFO order

# What Is a Subroutine?

- Group of instructions
- Written separately from the main program
- Performs a repeated subtask
- Called whenever it is needed
- Returns to the main program afterwards

## WHY USE IT

No repeated code

Memory used efficiently

Large project → modules

Developed independently

Registers protected by the stack

# CALL and RET

## CALL 16-bit address

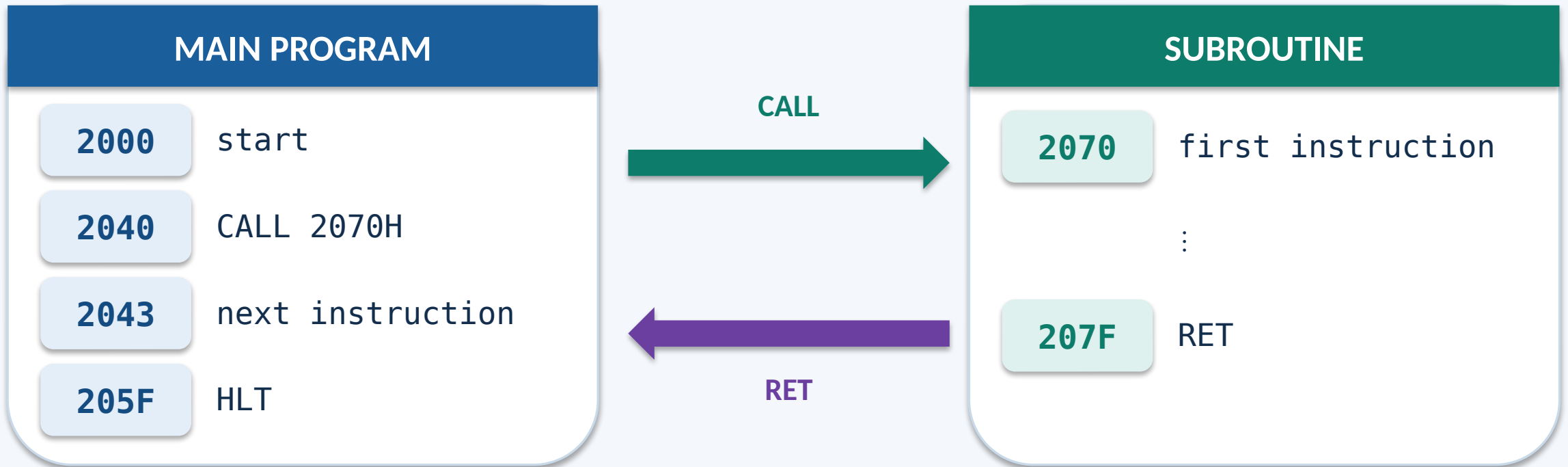
- 3-byte instruction
- Saves the program counter on the stack
- Address of the next instruction
- Stack pointer decremented by 2
- Jumps to the subroutine address

## RET

- 1-byte instruction
- Top two stack bytes → program counter
- Stack pointer incremented by 2
- Returns unconditionally
- Placed at the end of the subroutine

**CALL in the main program • RET in the subroutine — always paired**

# The Round Trip



**SP initialized at 2400H • 2043H stored at 23FE-23FF • RET brings it back**

## Inside CALL — 5 Machine Cycles · 18 T-States

| CYCLE | TYPE         | WHAT HAPPENS                   |
|-------|--------------|--------------------------------|
| M1    | Opcode Fetch | Fetch CD · SP → 23FFH          |
| M2    | Memory Read  | 70H → register Z               |
| M3    | Memory Read  | 20H → register W · PC = 2043H  |
| M4    | Memory Write | PCH = 20H → 23FFH · SP → 23FEH |
| M5    | Memory Write | PCL = 43H → 23FEH              |

Next instruction cycle: W / Z (2070H) → address bus

# Inside RET — Coming Home

| CYCLE | TYPE                | WHAT HAPPENS                  |
|-------|---------------------|-------------------------------|
| M1    | Opcode Fetch        | Fetch C9                      |
| M2    | Memory Read (stack) | 43H → register Z · SP → 23FFH |
| M3    | Memory Read (stack) | 20H → register W · SP → 2400H |

W / Z (2043H) → address bus

Main program resumes

Stack pointer is back where it started — nothing leaked

# Traffic Signal Controller

| SIGNAL     | BIT | ON TIME |
|------------|-----|---------|
| Green      | D0  | 15 s    |
| Yellow     | D1  | 5 s     |
| Red        | D2  | 20 s    |
| WALK       | D6  | 15 s    |
| DON'T WALK | D7  | 25 s    |

**Total sequence = 40 seconds**

## HOW IT IS BUILT

Bit patterns: 41H · 84H · 90H

1-second DELAY subroutine

Count passed in register B

CALL DELAY three times

PUSH / POP protect D, E, A, flags

JMP START repeats the cycle

## RST Instructions

**RST 0**    **0000H****RST 1**    **0008H****RST 2**    **0010H****RST 3**    **0018H****RST 4**    **0020H****RST 5**    **0028H****RST 6**    **0030H****RST 7**    **0038H**

- 1-byte Call instructions
- Transfer execution to page 00H
- Program counter saved on the stack — exactly like CALL
- Used with the interrupt process (Chapter 12)

## Call and Return — Four Flags

| FLAG   | CALL if SET | CALL if RESET | RETURN if SET | RETURN if RESET |
|--------|-------------|---------------|---------------|-----------------|
| Carry  | CC          | CNC           | RC            | RNC             |
| Zero   | CZ          | CNZ           | RZ            | RNZ             |
| Sign   | CM          | CP            | RM            | RP              |
| Parity | CPE         | CP0           | RPE           | RP0             |

Condition met → transfer

Not met → continue in sequence

**Eight conditional Calls • eight conditional Returns • four flags**

# Advanced Subroutine Concepts

## Multiple-Calling

One subroutine

Called from many places

Return address on the stack

## Nesting

A subroutine calls another

All return addresses stacked

Limited by stack space

## Multiple-Ending

One CALL, several exits

RZ • RC • RET

Conditional returns

**Every one of them works because the stack unwinds in LIFO order**

# CALL / RET vs. PUSH / POP

|                   | CALL / RET                     | PUSH / POP                |
|-------------------|--------------------------------|---------------------------|
| Who stores it     | The microprocessor — automatic | The programmer — explicit |
| On store          | SP - 2                         | SP - 2                    |
| On retrieve       | Top two → Program Counter      | Top two → Register pair   |
| After retrieve    | SP + 2                         | SP + 2                    |
| Conditional forms | 8 Call + 8 Return              | None                      |

**Same stack • same LIFO • different owner**

# Thank You

---

## Stack

memory in R/W

## Stack Pointer

16-bit register

## PUSH / POP

the programmer's tools

## CALL / RET

the processor's tools

## LIFO

the rule behind everything

## Subroutine

reuse and modularity

## Questions?